

桌伴 DeskMate —— 作品介绍

openvela AI 大赛 2026 · 参赛编号 contest2026_342 · 队伍 ez-xu 赛道：新硬件平台适配（ESP32-P4X 待适配板）+ AI 硬件产品创新（双赛道） 版本：2026-09-18（提交版） · 源稿：本文（docs/07_作品介绍.md） 说明：本文如实记录已完成、未完成与已知限制，全部声明均可由仓库内证据复现核验。

一、一句话介绍

桌伴（DeskMate）是一块基于 openvela + ai_agent 的 7 英寸智能视觉中控屏：它能看见、会主动、会执行——摄像头看到你进门就主动问候，早上到点自动播报天气与日程，说一句"记一下明早 9 点开会"就记下并到点提醒，设备异常主动告警。

与"问答式"语音助手不同，桌伴的重点是**主动**：它不需要你先开口。

二、赛题四点要求对齐（最终状态）

官方要求	本作品交付	状态
① Agent 在硬件设备上跑起来	openvela 移植到 ESP32-P4X-C5-Function-EV-Board （官方待适配板，RISC-V 双核 400MHz + 32MB PSRAM + RJ45 以太网 + MIPI-DSI/CSI）；ai_agent 完整启动 P0→P6；36 工具 / 12 Skill；eth0 DHCP 联网； LLM 端到端对话真机实测通过	真机

官方要求	本作品交付	状态
② 至少 1 个自定义 Skill	中控助手（center-assistant）、速记工单（quick-note）—— 面向本作品场景自研，开机写入 /data/ai_agent/skills/	真机 (1 2 built-in)
③ 至少 1 个"主动+执行"场景	定时主动 ：cron 真实运行（作业表移 PSRAM），到点自动执行； 事件主动（视觉） ：MIPI-CSI SC2336 驱动链路已打通（识别/开流/DMA 零错误），出帧卡在 MIPI 数据 lane 物理通路 （见 § 六.4）	cron ； 摄像头待换模组
④ 交付完整应用场景说	本文 + docs/ 全套（规划/需求/框架/设计/测试/复盘/状态/踩坑笔记 7 篇）	

三、用户故事

独居上班族小林回到书桌前。桌伴的摄像头识别到画面变化，屏幕与语音主动问候："欢迎回家，今天24°C，晚间有雨。" 他说"帮我记一下明早9点项目评审"——桌伴把它记成一张工单并自动建了一个明早9:00的提醒任务。第二天早上，桌伴到点主动提醒他。他全程没有打开任何App。

四、作了哪些工作

4.1 工作全景（8/6 → 9/18, 44天, 130次提交）

阶段	时间	内容	结果
环境搭建	8/6		
	/		
	6/8	openvela 工作区搭建；goldfish 模拟器跑通；	
	8/8	ai_agent 模拟器验证	
	/		
	1/2		
esp32p4移植	8/8		
	/		
	1/2		
	2/8	公版驱动复制（NuttX 上游 esp32p4 基线） +	
	-	openvela BSP 适配	
	8/8		
真机烧	/		
	1/8	WDT 复位循环 / USB-Serial 流控 / 堆双重分配 三	
		个根因；console + EMAC 打通	
	1		

阶段	时间	内容	结果
录 攻 坚	8		
	-		
	8		
	/		
	2		
网 络 攻 坚	4		
	8		
	/		
	3	eth0 未注册 → EMAC DMA 缓冲过大	
	0	(NRXDESC/NTXDESC 20→4) + CLIC 中断使能	
ai_ ag ent 上 板	-	缺陷	
	9		
	/		
	1		
	9		
主 线 闭 环	/		
	1	OOM (栈裁剪 + FS_TMPFS 修复) ; PSRAM 写	
	-	hang 深度调试 → bootloader PMP 时序缺陷	
	9		
	/		
工 程 化 交 付	4		
	9	ai_agent 完整启动成功 (OpenOCD 断点/栈分析	
	/	定位)	
	4		
	9	三仓改动收拢 + manifest copyfile 自动映射 +	
	/	export/deploy 工具 + PR #7 合入 ; API key 泄露	
	5	事件处置	
	-		
	9		
	/		
	1		

阶段	时间	内容	结果
SRAM 空间攻坚	0		
	9		
	/		
	1		
	0	512KB SRAM 装不下代码 → MCUboot 二级引导	
	-	+ flash XIP （代码迁入 16MB flash），SRAM 静态占用 460KB → 120.3KB	
	9		
	/		
	1		
	1		
外设落地	9		
	/		
	1		
	1		
	-	显示 / 触摸 / LVGL / DNS / 摄像头，逐一打穿并	
	9	修复多个平台级驱动缺陷	
	/		
	1		
	5		
	验证与修复	9	LVGL 真机渲染核实（JTAG 帧缓冲导出）； "ai_agent 一启动整机失聪" 定位并修复；摄像头定论
/			
1			
7			
9			
端到端打通	/		
	1	ai_agent LLM 端到端对话 （三个独立缺陷：端点 / TCP backlog / 计时被 TLS 改时钟）	
	7		
	晚		
收	9	免部署复现（325 条 copyfile）；演示视频 + 真	

阶段	时间	内容	结果
尾	/		
提	1	机证据日志；AI 日志补导 28 会话；本文	
交	8		

4.2 核心工作量说明（技术难度主攻）

- 1. **完整移植**：把 NuttX 上游 esp32p4 平台移植进 openvela（板级 + 芯片层 + 驱动），以 4 棵文件树 + 325 条 manifest <copyfile> 交付，评审 repo sync 即得完整工作树，无任何部署步骤。
- 2. **代码空间硬限制突破（本项目最硬的一战）**：
 - SIMPLE_BOOT 模式下代码只能跑在 512KB SRAM 窗口内，实测占用 460KB+，仅余几十 KB；
 - 证据链证明窗口**不可扩容**（ROM 保留区 + ROM 锁定 PMP + XIP-from-PSRAM 为死代码）；
 - **解法**：切 MCUboot 二级引导，text+rodata（约 400KB）迁入 16MB flash 以 XIP 执行 → SRAM 静态占用 **460KB → 120.3KB**（9/17 干净树复测）；
 - 为此移植 MCUboot 的 esp32p4 端口（上游无此板支持），修 ABI/工具链/头文件五类障碍（固化为幂等补丁），并自研两个工具让 NuttX 应用能被 MCUboot 加载（mcuboot_mkloadhdr.py、mcuboot_p4_loader_patch.py：映射 IROM + 使能 L1 cache 总线）。
- 3. **空间释放后恢复被裁剪的 AI 能力**：web_search、llm_vision/tool_vision、cron 从弱桩变为真实实现。
- 4. **外设逐项落地**：MIPI-DSI 7" 屏（EK79007）+ GT911 触摸 + LVGL 触控 UI、摄像头（SC2336/CSI/DW-GDMA）、DNS/DHCP 全自动联网、PSRAM 33.9MB 可用。

4.3 AI 开发方式

- **全流程 AI Coding**：从移植到驱动调试到文档，均由 AI 辅助完成；对话日志导出至 logs/（claude-code + dsh 双工具，28 会话 / 35,831 事件，logs/ez-xu/manifest.json 汇总，DSH 会话经 tools/engineering/dsh_session_to_contest_log.py 转换为赛事日志格式）。
- **技能沉淀**：使用 openvela 官方 AI 开发技能集（17 个，工作区 .claude/skills/）；同时 manifest 登记自有技能仓 ez-xu/openvela-skills → 工作区 .agents/（19 个

skills, repo sync 自动拉取，评审可直接阅读）。

五、最终输出哪些东西

5.1 交付物清单（全部已入仓）

交付项	位置	说明
作品代码	4 棵文件树 + manifest 325 条 copyfile	nuttx/、board/esp32p4/、app/apps/、app/ai_agent/；评审 repo init → sync → build 三步复现， 无部署步骤
可执行固件	构建产出 双镜像	应用 nuttx.bin 1,835,008 B（MCUboot 签名）+ MCUboot 二级引导 24,672 B；一次 build.sh 全出
演示视频	提交材料/ 6360b48f....mp4	27.7s • HEVC(H.265) • 720×1280 竖屏 • 含音轨 ，板端真机演示录像（≤5 分钟要求）
真机证据日志	提交材料/*.log	boot_nsh/aiagent_milestones/ aiagent_live/lvgldemo/ ws_llm_endtoend (9/18 实测)
×5		
A	logs/ez-xu/	28 会话 / 35,831 事件（8/21-9/17，含脱敏校验）
I		

交付项

Coding

日志

文档体系

闭环工具

AI开发技能资产

专属仓库

位置	说明
docs/ + README.md + tools/README.md	规划 01 / 需求风险 02 / 工程框架 03 / 模块设计 04 / 功能闭环测试 05 / 开发复盘 06 (27 节) / 状态 STATUS / 踩坑笔记 ×7 / 验证截图
tools/ (三分类)	engineering/ (export、manifest 校验、revision 锁定、日志转换)、build_flash/ (两镜像烧录、一键演示)、test/ (串口 harness、里程碑断言、摄像头/失聪诊断、帧缓冲截图、WS 对话、视频合成)
manifest → 工作区 .agents/	openvela-skills (19 个 skills)
https:// github.com/ open-vela/ contest2026_342 _ezdezhandui	全部提交经 fork + PR (当前 PR #12 , MERGEABLE)

5.2 真机已闭环的能力（有串口/截图/JTAG 证据）

能力	证据
MCUboot 二级引导 + flash XIP	启动日志 Mapped IROM vaddr=0x40000000 paddr=0x30000 (docs/demo/01_boot_mcuboot.log)
console + eth0 DHCP 联网	/dev/ttyUSB0; eth0 192.168.1.105 RUNNING (主机同网段 ICMP 通) ; DNS 可用
PSRAM 32MB	free: Umem total ≈ 33.9 MB
ai_agent 完 整启动	P0→P6 全 rc=0; 36 tools / 12 skills (含自研 ×2) ; WS 28789 监听
LLM 端到端 对话	WS 28789 → 消息 → LLM → 真实回答 (提交材料/ws_llm_endtoend.log、logs/verify-2026-09-17/agent_llm_ws.txt)
cron 定时主 动	[cron] Cron started (作业表移 PSRAM 堆)
LVGL 触控 UI	/dev/fb0 1024x600 RGB565 (EK79007) + /dev/input0 (GT911) ; 真机渲染经 JTAG 帧缓冲导出核实 (docs/验证截图_LVGL 界面_1024x600.png)
摄像头驱动 链路	SC2336 识别 0xcb3a、CSI 2 lane + DMA 武装、零错误 (出帧受限见 § 六.4)

六、中间遇到哪些问题，怎么解决（诚实记录）

完整过程见 docs/06_开发过程复盘与改进清单.md（27 节）与 docs/踩坑笔记_01~07。以下按"影响面从大到小"列出，已解决与未解决分开。

6.1 已修复的主要缺陷（按影响面排序）

A. 平台级驱动/内核缺陷（影响面大于本项目，9 个）

#	缺陷	根因	修复
1	ai_agent 启动整机失败" (9/17 定位，内核级)	riscv_doirq() 的早期引导保护 if (g_running_task == NULL) return regs; 把所有 trap 吞掉，而 up_exit() 故意先置 NULL 再发 ECALL(SYS_restore_context) → 该 ECALL 被无限重发（实测 ≈49 万次/秒 ECALL 风暴），CPU 100% 卡 trap，节拍中断/网络/串口全哑；与 agent 无关：子 shell 敲 exit 同样死机	保护只对硬件中断生效 (irq > RISCV_MAX_EXCEPT ION)；修复后 agent 启动完成 ping 5/6、TCP 28789 通 (logs/verify-2026-09-17/wedge_fix.txt)
2	板级 Kconfig 静默失效 (9/11)	残留 0 字节 boards/dummy/Kconfig 使板级 Kconfig 永不生效（影响所有 esp32p4 板）	修正/清理 Kconfig；该板所有板级选项恢复生效
3	UART 控制台输出丢字节 (9/11)	esp_lowputc_send_byte() 未等 TX FIFO 空间即写，突发静默丢字节（实测吞 ~1.5KB 启动日志；影响所有 espressif 串口控制台）	等待 FIFO 空间 + CONFIG_UART0_TXB UFSIZE 提到 2048
4	GT911 触摸屏驱	gt9xx.c 的 ioctl = NULL → TSIOC_GETMAXPOINTS 返回 -ENOTTY，LVGL 拒绝打开	补 ioctl；后另发现产品 ID 校验吃掉 NUL 补齐字节 (9/14) —

#	缺陷	根因	修复
5	动缺 ioctl l (9 /11)	/dev/input0	并修复
	V4L2 上层 拒绝 裸 Bayer (9/ 11)	v4l2_cap.c 格式白名单无裸 Bayer → 无 ISP 的 CSI 传感器无法出流	显式声明 g_sc2336_frmintervals[] (30fps) 并行 RAW10
	MIPI- DSI 显示 不亮 (9/ 14)	mipi_dsi_device.c 8 处 struct mipi_dsi_msg 未零初始化 → 一条 DCS 写被当成读发 → 等 BTA 应答超时 -110	零初始化结构体；真机 /dev/fb0 ready 1024x600 RGB565
	任务 退出 → 整机死 机		
	7 (后 台命令 cmd & 必崩)	openvela 自身引入的回归 (与 LVGL 无关)	定位为上游回归并绕过 (docs/06 § 23.10)
8	摄像头 DW- GDM	DSI 非共享独占把最后一条空闲 CLIC 线打上 NONSHARED → CSI 摄像头共享申请被拒 (初始化 - 5)	DSI 改共享注册 (ESP_INTR_FLAG_SHARED) ; 真机 Camera registered on

#	缺陷	根因	修复
	A 中断 (9/15)		/dev/video0
9	up_enable_irq() 从不置位 CLIC IE (9/15)	esp_setup_irq_intrstatus() 总带 INTRDISABLED → esp_intr_alloc 置的 IE 又被清掉; esp_intr_enable() 只重连路由矩阵不写 IE	up_enable_irq() 补 esp_intr_enable_source(真实 intno) (影响面很大)

B. 配置与工具链路问题（本项目侧，已解决）

#	问题	根因	修复
10	DNS 不可用 (9/15)	三个叠加: CONFIG_NET_UDP 未开、CONFIG_NETINIT_DHCP 未开 (与 NETUTILS_DHCP 区分)、netinit 不等链路就发 DHCP	全开 + 等待链路; 现 DHCP 全自动, nslookup → 202.69.4.22; MTU 576→1500
11	烧录 /	旧单镜像流程失效; MCUboot 拒收 TCM 目标; 引导器重建后	两镜像烧录固化 usb_stable.sh; TCM 校验放行改由 patch 脚本

#	问题	根因	修复
	引导链路	启动循环	承载 (2d2e99d)

6.2 关键工程难题（已解决）

难题	根因	解法
PSRAM 写入 hanging (9/3-9/4, 40+项排除)	bootloader PMP 时序缺陷（非硬件 bug）：引导器在 PSRAM 初始化前锁了 PMP	跳过 bootloader 的 PMP 设置（ <code>#if 0</code> 固化）；OpenOCD 决定性诊断；PSRAM 32MB 完全可用
ai_agent 断连OOM / assert (9	系统 heap 仅 28KB，agent 主栈过大； <code>/data</code> 挂载失败引发配置层崩溃	主栈 16KB + EMAC DMA 降载 + <code>CONFIG_FS_TMPFS</code> （修挂载）

难题

根因

解法

/1
)

SRAM 装不下代码

(9
/10
-
9/1
1)

API key 泄露事件

(9
/7
发
现
、9
/10
处
置
)

re

512KB SRAM 窗口硬限制，实测 460KB+

key 编在 agent_config.h 随代码提交

清单里移除的 copyfile 目标会被 repo 删除

MCUboot 二级引导
+ flash XIP（见
§ 4.2），SRAM →
120.3KB

移除内嵌 key →
agent_secrets.h
（.gitignore 忽略）
注入机制；历史重
写；后续日志导出
又发现 4 处残留，
已脱敏
(5663907)；△
平台侧轮换仍未完
成（见 § 六.4）

export.py 加"目录

难题

根因

解法

posyncnc
会删除落位文件
(9/18)

条目"守卫 + 记录事故 (7b0c23d)

LLM
端到端不通
(9/17
晚，
三个
个
独立
缺陷
)

① 端点不对：key 属于 Token Plan 网关
(token-plan-cn.xiaomimimo.com，默认
api.xiaomimimo.com 401) ； ②
CONFIG_NET_TCPBACKLOG 未开 → 连接被
RST； ③ 计时用了会被 TLS 改写的墙上时钟 →
好答案被当超时丢掉

三处全修 → 真机端
到端对话成功
(agent_llm_ws.txt)

6.3 开发方法论上的坑（如实记录）

- 先做对照组：多次被"现象"带偏（如先被"没喂狗"带偏、先判"LCD 模组硬件坏"——真

因是驱动缺陷）。

- **halt 伪迹**：JTAG halt 期间读寄存器会得到"假现场"；帧缓冲导出会冻结画面。
- **症状 ≠ 病因**："整机失聪"先怀疑 agent 逻辑，判别实验（子 shell `exit` 同样死机）才转向内核。
- **生成文件陷阱**：`esp_loader.c` 是构建时生成的文件，手改会被覆盖 → 修复必须落在生成器（patch 脚本）。
- 以上已固化为 `docs/06` § 26.8 的方法学教训。

6.4 未解决 / 需硬件动作（诚实说明）

项	状态	说明与复测方法
摄像头出帧	驱动链路通、物理链路无数据	SC2336 识别 / 开流 / DMA 武装全零错误，但桥 FIFO 零字节、D-PHY 数据 lane stopstate=0（时钟 lane 活）→ 故障收敛到 MIPI 数据 lane 物理通路 （模组/排线/连接器）。上游 issue #53 同症状，换模组后恢复。重插/更换后一条命令复测： <code>tools/test/camera_diag.sh 35</code> （输出 <code>MOVING</code> 即出帧）
LLM问答延迟	⚠️ 可用但慢	一次端到端问答实测约 261s （推理模型 + 15KB 工具 schema 固有开销）；体验偏慢，非功能缺陷
Monkey	⚠️ 必须做	key 曾两次误入公开仓历史（已从仓库脱敏/移除），但 仅删除不能使其失效 ，须在平台侧吊销/轮换；新 key 经工作区 <code>packages/ai_agent/include/agent_secrets.h</code> 注入（不入仓），需重新构建烧录


```

# 可自选检: python3
contest2026_342_ezdezhandui/tools/engineering/gen_manifest_copyfiles.py --check

# 2. 构建（一条命令产出双镜像）

./build.sh vendor/espressif/boards/esp32p4/esp32p4-function-ev-board/configs/nsh --
cmake -j8

# → cmake_out/esp32p4-function-ev-board_nsh/nuttx.bin（应用）+ nuttx/mcuboot-
esp32p4.bin（引导）

# 3. 烧录（△ MCUboot 两镜像：0x20000 引导 + 0x200000 应用）

cd contest2026_342_ezdezhandui && tools/build_flash/usb_stable.sh

# 4. 验证

python3 tools/test/board.py reset # 复位并抓启动日志

tools/test/verify_aiagent.sh # P0~P6 + 12 skills + eth0 + cron 断言

python3 tools/test/ws_demo.py --host <板 IP> --port 28789 --send "只回复: 你好" --wait
280

tools/test/camera_diag.sh # 摄像头链路一次性判定

```

实测构建耗时（2026-09-18 干净树，本机）：依赖首次拉取 30~60 分钟（esp-hal-3rdparty ≈594MB + MCUboot 子模块 ≈1.6GB）；依赖就绪后编译 2,426 步 **6 分 32 秒**。

八、评分对照（官方 30/20/20/10/10/10）

评分项	本作品对应
技术难度	ESP32-P4X 新平台移植 + MCUboot 二级引导 + flash XIP + 9 个平台级驱动/内核缺陷修复 (另 2 个配置/工具链路问题，见 § 六.1；其中"整机失聪"为内核级)
30	
产品创新性	桌伴「能主动、会执行」产品叙事：ai_agent + 自定义 Skill ×2 + cron 定时主动；离线可用设计
20	
项目完整度	代码 + 可复现 Demo（三步复现）+ 文档齐备 + 真机证据逐项归档
20	
AI 开发	全流程 AI Coding：28 会话 / 35,831 事件日志 + .agents/ 技能资产 + docs 沉淀
10	

评分项

本作品对应

0

商业潜力

桌面陪伴/中控真实场景（可视门铃/看护/中控三合一叙事）；P4X 官方定位场景

1

0

展示效果

演示视频（27.7s 板端真机录像）+ 本文 + 答辩（S 七复现路径可现场走一遍）

1

0

九、总结（一句话实话实说）

做到了：把 openvela 完整移植到一块官方待适配的 ESP32-P4X 板上，突破了 SRAM 装不下代码的硬限制（MCUboot + flash XIP），修掉了 9 个平台级驱动/内核缺陷（另修 2 个配置/工具链路问题），最终在真机上跑通了 "显示 + 触摸 + 联网 + LLM 端到端对话 + 定时主动 + 自定义 Skill" 的完整桌伴主干，并以 "零部署步骤" 的 manifest 交付方式让评审三步复现。

没做到：摄像头出帧（卡在 MIPI 数据 lane 物理通路，需换模组/排线）、语音链路（时间盒取舍）、MiMo key 的平台侧轮换（需要你操作）。